

Big Data Aplicado

*Conforme a contenidos del «Curso de Especialización
en Inteligencia Artificial y Big Data»*



Universidad de Castilla-La Mancha

Escuela Superior de Informática
Ciudad Real

3

Capítulo

Gestión de Soluciones - III

Julio Alberto López Gómez

Este capítulo profundiza en dos soluciones de almacenamiento y gestión de datos propias de las bases de datos NoSQL: **las bases de datos XML y las bases de datos documentales**.

3.1. Bases de datos NoSQL

El término **NoSQL** proviene del inglés y está formado por las palabras “No” y “SQL”. El significado literal del término no debe confundir al lector ni alejarlo de su correcto significado. NoSQL no engloba solamente un conjunto de bases de datos y sistemas de almacenamiento que no utilizan SQL para su gestión. El significado de NoSQL amplía lo anterior, siendo el acrónimo de “*Not only SQL*”. Por tanto, NoSQL es una categoría general de sistemas de bases de datos que, además de SQL, utilizan otra serie de tecnologías para almacenar y gestionar los datos. El término NoSQL fue acuñado en 1998 por Carlo Strozzy y retomado en el año 2009 por Eric Evans, quien aprovechó este término para referirse a esta nueva generación de familias de bases de datos como “**Big Data**”. (ver [Tiw11]).

Las transacciones realizadas en los **sistemas de bases de datos relacionales** garantizan la consistencia y escalabilidad de las operaciones a través de una serie de características especificadas en el **modelo ACID**.

- **Atomicidad (A):** Garantiza que una transacción se ha realizado completamente o no, evitando que algunas operaciones se ejecuten a medias.
- **Consistencia (C):** También conocida como **integridad**, asegura que cualquier transacción realizada desde un estado seguro en la base de datos conducirá a esta a otro estado también seguro o válido.

- **Aislamiento (I: Isolation):** Permite que el resultado en el estado del sistema sea el mismo, independientemente del modo de ejecución de las transacciones (secuencial o concurrente).
- **Durabilidad (D): O persistencia,** garantiza que a pesar de que se produzca un fallo en el sistema, los cambios realizados por una transacción persistirán.

Alternativamente, las bases de datos **NoSQL** siguen el modelo conocido como **BASE**, más flexible que ACID en aquellos sistemas de bases de datos que no se adhieren estrictamente al modelo relacional.

- **Disponibilidad básica (Basic Availability):** El sistema siempre ofrece una respuesta a una petición de datos, aunque los datos sean inconsistentes, estén en fase de cambio o incluso se produzca un fallo.
- **Soft-state (S):** La consistencia de este tipo de bases de datos es eventual, por lo que el estado del sistema cambia a lo largo del tiempo, aunque no haya habido entradas de datos.
- **Eventual Consistency (E):** Cuando se dejan de recibir datos, el sistema se vuelve consistente. Así pues, los datos se propagan a todo el sistema, pero este continúa recibiendo datos sin evaluar la consistencia de los mismos para cada transacción antes de avanzar a la siguiente.

En la actualidad, las aplicaciones web modernas, el auge de la computación ubicua y el Big Data, presentan desafíos muy diferentes a los que presentan los sistemas de información tradicionales, implementados por medio de sistemas de bases de datos relacionales. Algunos **desafíos** son: el procesamiento masivo de datos, la alta frecuencia de lecturas y escrituras, los cambios dinámicos y frecuentes en el esquema de datos, la escalabilidad a costes razonables, gestión de datos temporales...En este contexto, en los últimos años han aparecido multitud de enfoques, modelos y tecnologías para aportar soluciones a los desafíos mencionados anteriormente. En este capítulo, se profundiza en **las bases de datos XML y las bases de datos documentales**.



Base de datos NoSQL: Sistema de bases de datos no relacional que utiliza distintas tecnologías para la gestión y almacenamiento distribuido de datos masivos sin un esquema estricto subyacente. Esta familia de base de datos permite incrementar la disponibilidad y escalabilidad del sistema, mejorando su rendimiento.

3.2. Bases de datos XML

El lenguaje **XML** (eXtensible Markup Language) es un lenguaje de marcas extensible derivado del lenguaje **SGML** (Standard Generalized Markup Language). Se trata de un lenguaje ideado para la definición y gestión de documentos que permite representar datos estructurados y semi-estructurados. En la actualidad, XML se ha convertido en un formato estándar para la comunicación entre aplicaciones, facilitando su integración. Entre las principales ventajas de XML destacan:

- **Datos autodocumentados:** Gracias a la posibilidad de definir cualquier tipo de etiquetas, no es necesario conocer el esquema para entender el significado de los datos.
- **Extensión:** Permite adaptar el lenguaje fácilmente a cualquier dominio.
- **Anidamiento:** La definición de estructuras anidadas dota a los documentos de gran flexibilidad para transferir y consultar información.
- **Flexibilidad:** Los documentos XML no tienen un formato estricto ni rígido, permitiendo añadir o ignorar información en ellos.

Estructura de un documento XML

La estructura de un documento XML está compuesta de elementos. Un **elemento** es una porción de documento delimitado por un par de etiquetas `<>` (apertura) y `< / >` (cierre) entre las cuales se incluye un texto llamado **contexto** del elemento. Un elemento sin contexto puede abreviarse mediante el uso de una única etiqueta `< nombre_elemento / >`. La estructura de un XML se forma **anidando** elementos para especificar la estructura del documento, formando una **estructura de árbol**. El fragmento de código 3.1 representa la estructura básica de un documento XML que representa un listado de asignaturas.

Listado 3.1: Listado de Asignaturas

```
1 <Asignaturas_Primer>
2   <Asignatura>
3     <titulo> Cálculo </titulo>
4     <tipo> Anual </tipo>
5     <profesor>
6       <nombre> Ana </nombre>
7       <apellido> Sanz </apellido>
8     </profesor>
9     <profesor>
10      <nombre> Roberto </nombre>
11      <apellido> Hernández </apellido>
12    </profesor>
13  </Asignatura>
14  <Asignatura>
15    <titulo> Física </titulo>
16    <tipo> Semestral </tipo>
17    <profesor>
```

```
18         <nombre> Carmelo </nombre>
19         <apellido> Moya </apellido>
20     </profesor>
21 </Asignatura>
22 </Asignaturas_Primer>
```

Los elementos de un documento pueden contener atributos. Los **atributos** se incluyen en la definición del elemento, dentro de la etiqueta de apertura. Sintácticamente, se representan mediante un par *nombre = valor*. A diferencia de un subelemento, **un atributo solo puede aparecer una vez en una etiqueta**.

El uso de atributos y su diferencia con respecto a los elementos no siempre es evidente. A **nivel de documento**, los atributos se introducen para añadir texto que no se visualizará en el documento, mientras que los subelementos formarán parte del contenido del documento. A **nivel de datos**, la diferencia es insustancial, puesto que la misma información se puede representar utilizando atributos o subelementos. Por ejemplo, el subelemento `< tipo >` de una asignatura, podría representarse como un atributo del elemento asignatura. El listado 3.2 representa la asignatura de Cálculo del ejemplo anterior, incluyendo el título de la asignatura como atributo de la misma.

Listado 3.2: Uso de atributos para la definición de asignaturas

```
1 <Asignaturas_Primer>
2   <Asignatura>
3     <titulo> Cálculo tipo = 'anual' </titulo>
4     <tipo> Anual </tipo>
5     <profesor>
6       <nombre> Ana </nombre>
7       <apellido> Sanz </apellido>
8     </profesor>
9   </profesor>
10   <nombre> Roberto </nombre>
11   <apellido> Hernández </apellido>
12 </profesor>
13 </Asignaturas_Primer>
```

Dado que una de las principales aplicaciones de XML es el intercambio de información entre organizaciones, y ante la flexibilidad del lenguaje que permite definir cualquier nombre para una etiqueta, es posible que **existan conflictos entre los nombres de éstas**. Para ello, se recurre a los **espacios de nombres**, los cuales permiten a las organizaciones especificar nombres de etiquetas únicos. De esta forma, se añade al nombre de la etiqueta o atributo un identificador de recursos universal. El listado muestra de forma resumida un ejemplo de utilización del espacio de nombres para la definición de asignaturas.

Listado 3.3: Uso de un espacio de nombres

```
1 <Asignatura xmlns:AS="http://www.listado-asignaturas.com">
2   <AS:titulo> Cálculo tipo = 'anual' </AS:titulo>
3   <AS:tipo> Anual </AS:tipo>
```

```
4      <AS:profesor>
5          <AS:nombre> Ana </AS:nombre>
6          <AS:apellido> Sanz </AS:apellido>
7      </AS:profesor>
8      <AS:profesor>
9          <AS:nombre> Roberto </AS:nombre>
10         <AS:apellido> Hernández </AS:apellido>
11     </AS:profesor>
12 </Asignatura>
```

Al igual que ocurre en las bases de datos relacionales, las bases de datos XML presentan una serie de **lenguajes de definición de tipos de documentos XML**. Concretamente, a continuación se profundizarán en **DTD** y **XML Schema**. Seguidamente, se estudiarán los lenguajes **XPath**, **XSLT** y **XQuery**, como **lenguajes de manipulación de datos XML** para la definición y ejecución de consultas.

3.2.1. Definition Type Document (DTD)

En XML, se dice que un documento está **bien formado** si cumple con la sintaxis del lenguaje: correcta definición de elementos y atributos, correcto anidamiento... Como se puede apreciar, este concepto alude a la **dimensión sintáctica de los documentos**.

Por otra parte, se dice que un documento XML es **válido** cuando **está bien formado y cumple con la estructura de documento que ha sido dada a través de un lenguaje de definición de datos**. Esta definición de la estructura de los documentos XML es **opcional**, si bien es cierto que es recomendable, especialmente cuando se trabaja con documentos complejos y con un volumen grande de ellos. La definición de tipos de documento o **DTD permite definir la estructura de un documento XML**. Para ello, la DTD especifica la estructura de los elementos y atributos de un documento: qué elementos pueden aparecer, qué atributos, qué subelementos, multiplicidad de ellos... Existen multitud de validadores on-line de documentos XML y sus correspondientes DTDs y/o XML Schemas¹.

La **declaración de una DTD** dentro de un documento XML puede implementarse de dos formas: en primer lugar, de manera **interna**. En este caso, se añade la DTD después del preámbulo XML y justo antes del documento propiamente dicho. El listado 3.4 muestra un ejemplo de declaración interna de una DTD.

Listado 3.4: Declaración Interna DTD

```
1 <?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
2 <!DOCTYPE Elemento_Raiz [Aquí la DTD]>
3 <!--Aquí va el documento XML-->
```

¹<https://www.xmlvalidation.com>
<http://xmlvalidator.new-studio.org>

En segundo lugar, la **DTD puede declararse de forma externa**, definiendo la misma en un fichero .DTD aparte que después es enlazado en el documento. El listado 3.5 muestra un ejemplo de declaración externa de una DTD.

Listado 3.5: Declaración Externa DTD

```
1 <?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
2 <!DOCTYPE Elemento_Raiz SYSTEM "Mi_DTD.dtd">
3 <!-------Aquí va el documento XML----->
```

Siguiendo con el ejemplo del listado 3.1, el listado 3.6 muestra el archivo .dtd que validaría la estructura del documento de asignaturas.

Listado 3.6: DTD Listado de Asignaturas

```
1 <! DOCTYPE Listado_Asignaturas[
2 <! ELEMENT Asignaturas_Primeros (Asignatura+)>
3 <! ELEMENT Asignatura (titulo, tipo, profesor+)>
4 <! ELEMENT titulo (#PCDATA)>
5 <! ELEMENT tipo (#PCDATA)>
6 <! ELEMENT profesor (nombre, apellido)>
7 <! ELEMENT nombre (#PCDATA)>
8 <! ELEMENT apellido (#PCDATA)>
9 ]>
```

En el listado anterior, se define en primer lugar el tipo de documento “Listado_Asignaturas”. A continuación, y especificados entre corchetes, se describen los elementos que contendrá el documento, comenzando por el elemento raíz “Asignaturas_Primeros”. Este elemento estará formado por uno o más elementos “Asignatura” (especificado mediante el operador +). Seguidamente, el elemento “Asignatura” viene dado por un conjunto de tres subelementos: título, tipo y profesor. Además, cada asignatura puede tener uno o más profesores. Los elementos título y tipo vienen dados por cadenas de caracteres (denotadas con la instrucción #PCDATA) mientras que el elemento profesor estará formado por dos subelementos llamados “nombre” y “apellido” ambos definidos mediante cadenas de caracteres.

La definición de atributos para un elemento se hace a continuación de la definición del mismo utilizando `<!ATTLIST >`. El listado 3.7 muestra la DTD anterior en caso de que el tipo de asignatura sea considerado un atributo.

Listado 3.7: DTD Listado de Asignaturas con atributos

```
1 <! DOCTYPE Listado_Asignaturas[
2 <! ELEMENT Asignaturas_Primeros (Asignatura+)>
3 <! ELEMENT Asignatura (titulo, profesor+)>
4 <! ATTLIST tipo #CDATA REQUIRED>
5 <! ELEMENT titulo (#PCDATA)>
6 <! ELEMENT tipo (#PCDATA)>
7 <! ELEMENT profesor (nombre, apellido)>
8 <! ELEMENT nombre (#PCDATA)>
```

```
9  <! ELEMENT apellido (#PCDATA)>
10 ]>
```

Finalmente, la tabla 3.1 muestra un resumen de definición de elementos y atributos en una DTD.

Tabla 3.1: Descripción DTD de elementos y atributos XML

ELEMENTO			ATRIBUTO		
Tipos	#PCDATA	Cadena de caracteres	Tipos	CDATA	Cadena de caracteres
	EMPTY	Elemento sin contexto		ID	Identificador
	ANY	Cualquier tipo		IDREF	Referencia a un ID
		Disyunción		IDREFS	Referencia a múltiples ID.
Operadores	+	Uno o más	Valores	#REQUIRED	Obligatorio
	*	Cero o más		#IMPLIED	Opcional
	?	Cero o uno		#FIXED	Valor fijo
				#DEFAULT	Valor por defecto

3.2.2. XML Schema

Se trata de un **lenguaje de definición de la estructura de documentos XML más sofisticado que DTD**. Aunque es **más complejo, soluciona muchos de los inconvenientes de las DTD**. De hecho, XML Schema es un superconjunto del lenguaje DTD especificado mediante la sintaxis de XML. Entre otros aspectos, XML Schema soporta no solo el tipo cadena de caracteres (soportado por DTD), sino también tipos numéricos, tipos complejos como listas y además permite al usuario definir sus propios tipos así como extender tipos de datos complejos mediante un mecanismo similar a la herencia. Además, XML Schema soporta el concepto de espacios de nombre, permitiendo reutilizar elementos de un esquema en otros.

Un XML Schema se especifica en un documento aparte del documento XML con extensión **.xsd**. Dentro de este archivo, que sigue las reglas sintácticas de cualquier documento XML, se especifican las definiciones de cada elemento del esquema, qué atributos incluye así como sus tipos de datos válidos asociados tanto a los elementos como a los atributos. De esta forma, **XML Schema permite definir un esquema para validar documentos XML, de forma más sofisticada a como lo hace DTD**. El listado 3.8 muestra un XML Schema para el ejemplo del listado de asinaturas.

Listado 3.8: XML Schema: Listado de asignaturas

```

1 <?xml version="1.0" encoding="utf-8"?>
2 <xs:schema attributeFormDefault="unqualified" elementFormDefault="qualified" xmlns:xs="
  http://www.w3.org/2001/XMLSchema">
3   <xs:element name="Asignaturas_Primer">
4     <xs:complexType>
5       <xs:sequence>
6         <xs:element maxOccurs="unbounded" name="Asignatura">
7           <xs:complexType>
8             <xs:sequence>
9               <xs:element name="titulo" type="xs:string" />
10              <xs:element name="tipo" type="xs:string" />
11              <xs:element maxOccurs="unbounded" name="profesor">
12                <xs:complexType>
13                  <xs:sequence>
14                    <xs:element name="nombre" type="xs:string" />
15                    <xs:element name="apellido" type="xs:string" />
16                  </xs:sequence>
17                </xs:complexType>
18              </xs:element>
19            </xs:sequence>
20          </xs:complexType>
21        </xs:element>
22      </xs:sequence>
23    </xs:complexType>
24  </xs:element>
25 </xs:schema>

```

El listado anterior comienza definiendo un XML Schema en la segunda línea de código. Cualquier XML Schema deberá incluir esta línea de código. El atributo *attributeFormDefault* = “*unqualified*” indica que no es necesario utilizar el prefijo del espacio de nombres para definir los atributos del esquema, mientras que el atributo *elementFormDefault* = “*qualified*” indica que no es necesario utilizar el prefijo del espacio de nombre para definir los elementos del esquema. A continuación, se define el elemento raíz “Asignaturas_Primer” el cual se trata de un tipo complejo, puesto que no se define con un tipo de datos básico como entero, carácter, etc. Este elemento está formado por una secuencia de elementos del tipo “Asignatura” (el número máximo de elementos Asignatura no está limitado ya que *maxOccurs* = “*unbounded*”). Los elementos Asignatura, a su vez, están formados de una secuencia de elementos “nombre”, “tipo” (ambos de tipo cadena de caracteres) y un conjunto de elementos de tipo “profesor” no acotados, el cual se compone a su vez de los elementos “nombre” y “apellido”, ambos de tipo cadena de caracteres.

XML Schema permite representar la cardinalidad de un elemento, definiendo el número mínimo y máximo de ocurrencias mediante los atributos *minOccurs* y *maxOccurs*. Además, también es posible añadir restricciones de forma que se limiten los valores de un elemento o atributo, se permita elegir entre una lista de posibles... Finalmente, XML Schema dispone de los contenedores *sequence* para definir una secuencia ordenada de subelementos, *choice* para definir la elección entre posibles grupos de elementos o elementos y *all* para definir un conjunto no

ordenado de elementos. Al igual que sucedía con DTD, **existen distintas herramientas on-line que no solo permiten validar un documento XML a partir de su esquema XSD, sino que también permiten generar el esquema XSD a partir del archivo XML**².

3.2.3. XPath

En cualquier sistema de bases de datos, además de los lenguajes de definición de datos, los **lenguajes de manipulación de datos** son esenciales para poder **realizar consultas** y recuperar datos, realizar actualizaciones, etc.

XPath es un lenguaje de manipulación de datos XML. La principal característica de este lenguaje es que **interpreta un documento XML como una estructura de datos de tipo árbol**, donde los nodos y sus respectivas hojas se corresponden con los elementos y subelementos del documento XML. De esta forma, y utilizando comandos que permiten recorrer el documento de forma análoga a los comandos que se utilizan en una terminal de comandos para recorrer el sistema de archivos de un computador, es posible realizar consultas. La tabla 3.2 muestra un listado de los principales operadores utilizados en XPath.

Tabla 3.2: XPath: Operadores básicos

Operador	Significado
/	Navegación entre nodos del documento. Selecciona los nodos del nivel inferior
//	Navegación entre nodos del documento. Selecciona los nodos del nivel inferior de cualquier nodo especificado a continuación
.	Selecciona el nodo actual
..	Selecciona el nodo padre o nodo de nivel superior
	Expresa alternativas
@	Selecciona un atributo de un nodo
*	Selecciona todos los nodos
[]	Agrupar otros operadores

Con estos operadores, veamos cómo realizar algunas consultas de ejemplo sobre el fragmento XML que especifica un listado de asignaturas.

²<https://www.freeformatter.com>
<https://www.liquid-technologies.com/online-xsd-validator>,<https://extendsclass.com/xml-schema-validator.html>



Consulta 1. Recuperar los nombres de todas las asignaturas

```
/Asignaturas_Primer/Asignatura/titulo/text()
```



Consulta 2. ¿Cuántas asignaturas son impartidas por más de un profesor?

```
/Asignaturas_Primer/Asignatura[count(profesor)>1]
```



Consulta 3. ¿Quiénes son los profesores que imparten la asignatura de física?

```
/Asignaturas_Primer/Asignatura[titulo="Física"]/profesor/nombre
```

Es posible utilizar XPath de forma on-line a través de visualizadores³ o, por ejemplo, instalando el plug-in XPatherizer de Notepad++, en caso de utilizar este último editor de código XML.

3.2.4. XSLT

Otro lenguaje de manipulación de datos XML es el lenguaje XSLT. **El lenguaje XSLT (XML Stylesheet Transformations) proviene del lenguaje XSL (XML Stylesheet).** Este último permite especificar las opciones de formato de un documento XML en un archivo separado, aislando así la especificación del contenido y la presentación de un archivo XML. Podría decirse que XSL es a XML lo que CSS es a HTML. **El lenguaje XSLT permite especificar las opciones e formato de un documento XML, pudiendo transformar este documento en otros formatos como HTML, PDF, etc.** Dada la potencia de este lenguaje, que es capaz de transformar un archivo XML en otro que también pudiera ser XML, **se utiliza frecuentemente como lenguaje de consulta**

Las transformaciones XSLT se definen por medio de **plantillas**, que permiten a su vez recuperar contenido del documento XML a través de la utilización de expresiones XPath. La principal característica de XSLT es que la aplicación de plantillas se realiza de forma recursiva, lo cual recibe el nombre de **recursividad estructural**. El listado 3.9 presenta un ejemplo de aplicación de una transformación XSLT por medio de una plantilla.

³https://download.cnet.com/XPath-Visualizer/3000-7241_4-75804649.html,
<https://www.freeformatter.com/xpath-tester.html>

Listado 3.9: XSLT: Ejemplo de plantilla

```
1 <xsl: template match = /Asignaturas_Primer/Asignatura >
2 <titulo_asignatura>
3   <xsl:value-of select = titulo/>
4 </titulo_asignatura>
5 </xsl:template>
```

Tal y como se puede observar en el fragmento anterior, una transformación o consulta XSLT se define mediante dos órdenes básicas: *match* que especifica una expresión XPath para seleccionar uno o más nodos y *value – of select* que devuelve los valores especificados de los nodos que se han obtenido como resultado de la expresión XPath. Merece la pena destacar que **cualquier texto o etiqueta del archivo XSLT que no esté en el espacio de nombre se copia a la salida sin cambios**. Esto quiere decir, que el resultado del fragmento anterior no son únicamente los títulos de las asignaturas, sino estos mismos delimitados por una etiqueta de apertura y cierre llamada `< titulo_asignatura >`. De esta forma, a partir de un documento XML se ha generado otro con el resultado de la consulta. A modo de resumen, la tabla 3.3 define los principales constructores XSLT.

Tabla 3.3: Principales Constructores XSLT

Constructor XSLT	Significado
<code>< xsl : template match = “expresion XPath” ></code>	Selecciona nodos a devolver
<code>< xsl : value – of select = “valor” ></code>	Devuelve valores de los nodos seleccionados
<code>< /xsl : template >< xsl : template match = “*” / ></code>	Devuelve todos los nodos que no coinciden con alguna otra plantilla
<code>< xsl : attribute nombre = “nombre atributo” ></code>	Añade un atributo al elemento precedente
<code>< xsl : apply – templates/ ></code>	Aplica la recursividad estructural
<code>< xsl : keyname = “nombre clave” match = “elementos aplicar” use = “valor clave” / ></code>	Define una clave
<code>< xsl : value – of select = “key(“nombre clave” “valor”)”</code>	Devuelve los nodos que coincidan con el valor especificado. Especifica operaciones de combinación (JOINS)
<code>< xsl : sort select = “elemento o atributo” / ></code>	Devuelve los nodos resultados, ordenados por un elemento o atributo

Para trabajar con XSLT, se aconseja la descarga del software XML Copy Editor (<https://xml-copy-editor.sourceforge.io>).

3.2.5. XQuery

Es un **lenguaje de consulta de propósito general sobre datos XML estandarizado por el consorcio W3C**. XQuery procede del lenguaje de programación Quilt y, por este motivo, toma prestadas muchas características de otros lenguajes como XPath o SQL. De hecho, la sintaxis de XQuery es mucho más similar a la de SQL, al contrario que ocurría con lenguajes como XPath o XSLT.

Para definir una consulta en XQuery se construye una **expresión, denominada “FLWR”(flower, flor en inglés)**. A continuación, se describen cada una de las sentencias de las que se compone la expresión:

- **FOR:** Obtiene una serie de variables cuyos valores son el resultado de una expresión XPath. Es el equivalente a la cláusula **FROM** del lenguaje SQL.
- **LET:** Define y renombra expresiones complejas para ser utilizadas en el resto de la consulta, reduciendo la complejidad y aliviando la sintaxis. Es una cláusula **opcional**.
- **WHERE:** En esta cláusula, llamada igualmente en SQL, se especifican condiciones sobre los resultados obtenidos en la cláusula FOR.
- **RETURN:** Especifica y define el resultado que se va a obtener de la consulta. Se trata del equivalente a la cláusula **SELECT** en SQL.

Siguiendo la sintaxis anteriormente mencionada, el listado 3.10 muestra un ejemplo para obtener un listado con las asignaturas anuales.

Listado 3.10: XQuery: Ejemplo de consulta

```
1 for $x in /Asignaturas_Primer/Asignatura
2 let $nombre_asignatura:= $x/titulo
3 where $x/tipo='anual'
4 return <asignatura_anual> $nombre_asignatura </asignatura_anual>
```

En el listado anterior, la cláusula *for* permitirá recorrer todos los elementos asignatura. Por su parte, *let* define una expresión en la que se asigna a *nombre_asignatura* el título de la asignatura que se está recorriendo en cada momento. La cláusula *where* impone el criterio de que el tipo de asignatura sea anual y, finalmente, *result* define como resultado un fragmento de código XML donde el nombre de la asignatura anual (según la expresión indicada en *let*) aparecerá como resultado encerrado entre dos etiquetas XML llamadas *asignatura_anual*. Tal y como se puede apreciar, XQuery también permite generar nuevos documentos XML a partir de consultas.

Al igual que SQL, XQuery dispone de multitud de funciones para la consulta de datos. Así, es posible utilizar la función *distinct()* para eliminar duplicados en los resultados así como funciones de agregación como *sum()* o *count()* además de poder especificar funciones definidas por el usuario. XQuery también permite

utilizar “joins” incluyendo en la cláusula *where* el operador de combinación al igual que en SQL. Por otra parte, si bien es cierto que XQuery no posee el operador *group by* de SQL, este puede implementarse mediante la creación de consultas anidadas, las cuales sí que son soportadas por el lenguaje.

3.3. Bases de datos documentales: MongoDB

Los sistemas de bases de datos documentales u orientados a documentos son **sistemas de bases de datos NoSQL que almacenan datos, los cuales se estructuran en forma de documentos**. En los últimos años, han proliferado una gran variedad de sistemas de este tipo como pueden ser **Cassandra⁴, CouchDB⁵, Riak⁶ o MongoDB⁷**, sobre el cual trata esta sección.

El nombre **MongoDB** proviene de la palabra inglesa “**homongous**” que significa enorme. **MongoDB es, por tanto, un sistema de base de datos NoSQL, open source, orientado a documentos y escrito en lenguaje C++**. Este sistema de bases de datos está disponible no solo para múltiples plataformas y sistemas operativos (Windows, Linux, OS X) sino también como servicio empresarial en la nube y se puede integrar con otros servicios como, por ejemplo, Amazon Web Services (AWS).

3.3.1. MongoDB: Características y aplicaciones

Tal y como se ha presentado anteriormente, MongoDB es un sistema de bases de datos documental u orientado a documentos. Esta orientación hace que los datos se almacenen de forma estructurada en forma de documentos, los cuales se acoplan sin problema en los tipos de datos utilizados por los lenguajes de programación. Además, esta concepción hace que una base de datos MongoDB disponga de un esquema dinámico y fácilmente modificable.

En este apartado se pretende poner de relieve **las principales características de MongoDB** que hacen que sea en la actualidad uno de los sistemas de bases de datos NoSQL más utilizados tanto en el ámbito académico como en el profesional.

- **Alto rendimiento:** Gracias a la definición de los documentos y la creación de índices que hace que las lecturas y escrituras se realicen de forma más rápida.
- **Alta disponibilidad:** MongoDB dispone de servidores replicados con restablecimiento automático maestro.

⁴https://cassandra.apache.org/_/index.html

⁵<http://couchdb.apache.org>

⁶<https://riak.com>

⁷<https://www.mongodb.com/es>

- **Fácil escalabilidad:** Permitiendo distribuir colecciones de documentos entre diferentes máquinas de forma muy sencilla.
- **Indexación:** Similar al concepto de índice en una base de datos relacional, permite crear índices e índices secundarios para mejorar el rendimiento de las consultas.
- **Consultas ad-hoc:** Al igual que en una base de datos relacional, MongoDB da soporte a la búsqueda por campos, consultas de rangos, uso de expresiones regulares... A todo lo anterior, se añade la posibilidad de ejecutar y devolver una función JavaScript definida por el programador.
- **Replicación:** Siguiendo el modelo maestro-esclavo. El maestro puede ejecutar comandos de lectura y escritura mientras que el esclavo solo tiene acceso de lectura y la posibilidad de realizar copias de seguridad. En caso de que el maestro caiga, el esclavo puede elegir un nuevo maestro para mantener el servicio de replicación.
- **Balanceo de carga:** MongoDB es capaz de ejecutarse en múltiples servidores, pudiendo balancear la carga y/o duplicar los datos para mantener el correcto funcionamiento del sistema aunque se produzca un fallo hardware.
- **Almacenamiento de archivos:** Todo lo anterior, facilita que este sistema de base de datos pueda ser usado con un sistema de archivos GridFS con balanceo de carga y replicación.
- **Agregación:** Proporciona operadores de agregación y la posibilidad de utilizar funciones MapReduce para el procesamiento de datos por lotes.
- **Ejecución de Javascript del lado del servidor:** Es posible realizar consultas utilizando JavaScript, de forma que éstas son enviadas directamente a la base de datos para ser ejecutadas.

Todas estas características hacen que, en la actualidad, MongoDB sea uno de los principales sistemas de bases de datos NoSQL elegidos en multitud de aplicaciones como: **almacenamiento y registro de eventos, comercio electrónico, juegos, aplicaciones móviles, almacenes de datos, gestión de estadísticas en tiempo real y cualquier aplicación que requiera llevar a cabo analíticas sobre grandes volúmenes de datos.**

3.3.2. MongoDB: Conceptos básicos, utilidades y herramientas

Aunque MongoDB es un sistema de bases de datos NoSQL con todo lo que ello implica, también ofrece toda la funcionalidad de la que disponen las bases de datos relacionales. Sin embargo, **la estructura de una base de datos MongoDB difiere de la de una base de datos relacional.**

En un servidor MongoDB es posible crear tantas bases de datos como se desee. El concepto de **base de datos** es en este caso equivalente al de **base de datos** en los sistemas relacionales. Una vez creada, la base de datos estará compuesta por una o más **colecciones**. El término colección en el ámbito NoSQL es el equivalente al concepto de **tabla** en los sistemas relacionales. Cada colección, por tanto, estará formada por un conjunto de **documentos** (o incluso ninguno, en cuyo caso la colección estaría vacía). El concepto de documento en NoSQL se corresponde con el concepto de **registro** en una base de datos relacional. Un documento estará compuesto por una serie de **campos**, al igual que lo están los registros de una base de datos relacional. En el ámbito NoSQL, y más concretamente en MongoDB, cada documento viene dado por un archivo **JSON** (<http://www.json.org/>) en el cual, siguiendo una estructura clave-valor se especifican las características de cada documento. El listado 3.11 muestra un ejemplo de documento que define un barco.

Listado 3.11: Definición json de un documento barco

```
1 {  
2   name: 'USS Enterprise-D',  
3   operator: 'Starfleet',  
4   type: 'Explorer',  
5   class: 'Galaxy',  
6   crew: 750,  
7   codes: [10, 11, 12]  
8 }
```

Para la administración del sistema de bases de datos, MongoDB pone a disposición de los usuarios las siguientes utilidades:

- **mongo**: Se trata del shell interactivo de MongoDB que permite insertar, eliminar, actualizar datos y realizar consultas, además de replicar la información, apagar servidores y ejecutar código JavaScript.
- **mongostat**: Herramienta de línea de comandos que muestra las estadísticas de una instancia en ejecución de MongoDB
- **mongotop**: Herramienta de línea de comandos que muestra la cantidad de tiempo empleado en la lectura y escritura de datos por parte de la instancia en ejecución
- **mongosniff**: Herramienta de línea de comandos que permite hacer un rastreo del tráfico de la red que va desde y hacia MongoDB.
- **mongoimport/mongoexport**: Herramienta de línea de comandos para importar y exportar contenido en o desde .json, .csv o .tsv entre otros.
- **mongodump/mongorestore**: Herramienta de línea de comandos para la creación de una exportación binaria del contenido de la base de datos.

Aunque MongoDB tiene una interfaz administrativa accesible desde `http://localhost:28017/` (siempre que se haya iniciado con la opción `mongod --rest`, existen algunas herramientas gráficas para la administración y uso de este sistema de base de datos como **UMongo** (<https://github.com/agirbal/umongo>), el cual

es una aplicación open source de sobremesa para navegar y administrar un clúster MongoDB o **Robomongo** (<https://robomongo.org>), que también es una herramienta gráfica open source que incluye una terminal de comandos completamente compatible con el shell de mongo.

3.3.3. MongoDB: Operaciones CRUD

Cualquier sistema de datos permite definir operaciones básicas como son la creación de tablas e inserción de registros, lectura de datos, actualización y eliminación de registros. Estas operaciones básicas se conocen con el nombre de **CRUD (Create, Read, Update, Delete)**. A continuación, se muestran distintos ejemplos para ilustrar la implementación de estas operaciones en MongoDB.

Una vez iniciada una sesión del Shell de Mongo, es posible crear una base de datos utilizando el siguiente comando.

```
test>use DB_Ejemplo  
switched to db DB_Ejemplo
```

Para crear una colección de documentos “Barco” dentro de la base de datos DB_Ejemplo se puede escribir.

```
DB_Ejemplo>db.createCollection(“ships”)  
{ ok: 1 }
```

Para visualizar las bases de datos almacenadas en el servidor se puede utilizar el comando *show dbs* mientras que para obtener un listado de las colecciones de la base de datos sobre la que se está trabajando, es posible utilizar el comando *show collections*. Por otra parte, para insertar un documento dentro de la colección, como el mostrado en el listado 3.11, se ejecuta el siguiente comando.

```
db.ships.insert({ name:'USS-Enterprise-D',operator:'Starfleet',type:'Explorer',class:  
'Galaxy',crew:750,codes:[10,11,12]})
```

Para actualizar el nombre del barco “USS Prometheus” por “USS Something” es posible escribir el comando.

```
db.ships.update({ name : 'USS Prometheus' }, { name : 'USS Something' })
```

Mientras que si se pretenden establecer o cambiar varios atributos de un mismo documento, como por ejemplo el operador y la clase, se puede utilizar el comando *update* de la siguiente forma.

```
db.ships.update({name : 'USS Something'}, {$set : {operator : 'Starfleet', class : 'Prometheus'}})
```

Finalmente, la eliminación de algún atributo de un documento también es una operación de actualización que puede realizarse de la siguiente forma.

```
db.ships.update({name : 'USS Something'}, {$unset : {operator : 1}})
```

La eliminación de documentos de una colección puede realizarse de forma directa o bien utilizando expresiones regulares. A continuación se muestran dos comandos para ilustrar ambas formas de eliminación. El segundo de ellos elimina aquellos documentos que comienzan por “a”.

```
db.ships.remove({name : 'USS Prometheus'})  
db.ships.remove({name:{$regex:'A*'}})
```

Para leer y mostrar documentos, se utiliza el comando *find()*. A continuación se muestra un ejemplo en el que, el primer comando muestra un documento al azar de los existentes, el segundo muestra todos los documentos y lo hace de forma indexada en lugar de como texto seguido, el tercero muestra solo los nombres de los barcos y el último, encuentra un documento cuyo nombre sea “USS Defiant”.

```
db.ships.findOne()  
db.ships.find().prettyPrint()  
db.ships.find({}, {name:true})  
db.ships.findOne({'name':'USS Defiant'})
```

3.3.4. Mongo DB: Consultas y Agregación

Al igual que cualquier lenguaje de manipulación de datos, MongoDB dispone de las herramientas y operadores necesarios para realizar consultas sobre los documentos almacenados en las colecciones. A continuación, se muestran ejemplos de uso de los principales operadores.

Los operadores relaciones mayor que, menor que, mayor o igual que y menor o igual que se corresponden, respectivamente, con los operadores \$gt, \$lt, \$gte, \$lte. Además, también es posible utilizar el operador \$regex para recuperar elementos que cumplan una expresión regular. A continuación, se muestran dos consultas que recuperan aquellos barcos que permitan subir a bordo a más de cien pasajeros y otra consulta para recuperar aquellos que dejen subir tan solo a 100 pasajeros o menos.

```
db.ships.find({crew:{$gt:100}})

db.ships.find({crew:{$lte:100}})
```

El operador `$exists` permite devolver aquellos documentos para los que existe o no un determinado atributo. El siguiente comando permite encontrar aquellos barcos para los cuales existe el campo “class”.

```
db.ships.find({class:{$exists:true}})
```

Las funciones de agregación son especialmente útiles en los lenguajes de consulta y manipulación de datos. De esta forma, `$sum` permite agregar mediante la operación suma una serie de valores, `$avg` permite obtener la media, `$mnin` y `$max` encontrar el valor máximo y mínimo de un atributo para el conjunto de elementos, `$push` introduce en un array los resultados de la consulta que se ha realizado, `$addToSet` es similar al anterior solo que sin incluir valores duplicados y, por último, `$first` y `$last` permiten obtener el primer y último documento en una consulta. A continuación, se muestra un ejemplo del uso de cada uno de estos operadores de agregación.

```
db.ships.aggregate([{$group: {_id: "$operator", num_ships: {$sum: "$crew"}}}])

db.ships.aggregate([{$group : {_id : "$operator", num_ships : {$avg : "$crew"}}}])

db.ships.aggregate([{$group : {_id : "$operator", num_ships : {$min : "$crew"}}}])

db.ships.aggregate([{$group : {_id : "$operator", classes : {$push: "$class"}}}])

db.ships.aggregate([{$group : {_id : "$operator", classes : {$addToSet : "$class"}}}])

db.ships.aggregate([{$group: {_id: "$operator", last_class: {$last: "$class"}}}])
```

Finalmente, MongoDB pone a disposición de los usuarios multitud de funciones útiles en la realización de consultas. La tabla 3.4 muestra algunas de las más utilizadas.

Tabla 3.4: Funciones útiles en MongoDB

Función	Significado
<code>\$project</code>	Cambia el conjunto de documentos modificando sus claves y valores
<code>\$match</code>	Operación de filtrado para reducir el número de elementos recuperados
<code>\$group</code>	Operador de agregación para agrupar resultados
<code>\$sort</code>	Ordena los documentos
<code>\$skip</code>	Recupera los documentos a partir de un numero especificado por el usuario
<code>\$limit</code>	Limita los resultados de la consulta al valor pasado como parámetro a la función
<code>\$unwind</code>	Utilizado como equivalente al join de SQL

Bibliografía

- [AN95] Agnar Aamodt and Mads Nygård. Different roles and mutual dependencies of data, information, and knowledge — an ai perspective on their integration. *Data & Knowledge Engineering*, 16(3):191–222, 1995.
- [BELP13] Ulrik Brandes, Markus Eiglsperger, Jürgen Lerner, and Christian Pich. Graph markup language (graphml). In Roberto Tamassia, editor, *Handbook on Graph Drawing and Visualization*, pages 517–541. Chapman and Hall/CRC, 2013.
- [BNCD16] Rajkumar Buyya, Rodrigo N. Calheiros, and Amir Vahid Dastjerdi. *Big Data : Principles and Paradigms*. Elsevier Science & Technology, 50 Hampshire Street, 5th Floor, Cambridge. USA, 2016.
- [dMP99] Adoración de Miguel and Mario Piattini. *Fundamentos y Modelos de Bases de Datos*. Alfaomega, 1999.
- [FPSS96] Usama Fayyad, Gregory Piatetsky-Shapiro, and Padhraic Smyth. From data mining to knowledge discovery in databases. *AI magazine*, 17(3):37, 1996.
- [Fri19] J. Friesen. *Java XML and JSON: Document Processing for Java SE*. Apress, 2019.
- [GR09] Matteo Golfarelli and Stefano Rizzi. *Data Warehouse Design: Modern Principles and Methodologies*. Mcgraw-hill, 2009.
- [HT14] Francisco Herrera Triguero. *Inteligencia Artificial, Inteligencia Computacional y Big Data*. Servicio de publicaciones. Universidad de Jaén, Jaén, 2014.

- [IGG03] C. Imhoff, N. Galemme, and J.G. Geiger. *Mastering Data Warehouse Design: Relational and Dimensional Techniques*. Wiley, 2003.
- [Inm02] William H. Inmon. *Building the Data Warehouse, 3rd Edition*. John Wiley & Sons, Inc., USA, 3rd edition, 2002.
- [JVV03] Matthias Jarke, Yannis Vassiliou, Panos Vassiliadis, and Maurizio Lenzerini. *Fundamentals of Data Warehouses*. Springer-Verlag, Berlin, Heidelberg, 1st edition, 2003.
- [KR02] Ralph Kimball and Margy Ross. *The Data Warehouse Toolkit: The Complete Guide to Dimensional Modeling*. Wiley, 2 edition, April 2002.
- [MSC13] Cukier Mayer Schönberger, Viktor and Kenneth Cukier. *Big data. La revolución de los datos masivos*. Turner Publicaciones, Madrid, 2013.
- [MTK06] J. Mundy, W. Thornthwaite, and R. Kimball. *The Microsoft Data Warehouse Toolkit: With SQL Server 2005 and the Microsoft Business Intelligence Toolset*. Wiley, 2006.
- [PVMCV06] Mario Piattini Velthuis, Esperanza Marcos, Coral Calero, and Belén Vela. *Tecnología y diseño de bases de datos*. Ra-Ma, 2006.
- [RB02] Pablo Rodríguez Bocca. *Lenguajes canónicos para la descripción de grafos: Estudio y transformación entre esquemas de distintos modelos de datos*. Interoperabilidad, Monografía, 2002.
- [RWE15] Ian Robinson, Jim Webber, and Emil Eifrem. *Graph Databases*. O'Reilly, 2 edition, 2015.
- [SKS06] Abraham Silberschatz, Henry F. Korth, and Sudarshan S. *Fundamentos de bases de datos*. McGraw-Hill, Edificio Valrealty. 1 Planta. Basauri 17. 28023 Aravaca (Madrid), 2006.
- [Tiw11] Shashank Tiwari. *Professional NoSQL*. Wrox Press Ltd., 2011.
- [TR03] Erik T. Ray. *Learning XML*. O'Reilly, 2003.